



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

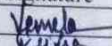
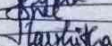
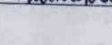
Section / Team Details

Section No.: 12

Team No.: 10

Project Title: Linux Signal-Based Process Communication and Control System

Team Members

Roll Number	Student Name	Signature
2520080018	Vennala	
2520080009	krishnasree	
2520080007	harshitha	

1. Abstract

The **Linux Signal-Based Process Communication and Control System** is an Operating Systems and System Programming project that demonstrates communication and control between processes using Linux signals. In a multitasking operating system, multiple processes may execute simultaneously and require an efficient mechanism to notify, coordinate, pause, resume, or terminate other processes. This project addresses this requirement by using Linux signals as a simple and lightweight method of inter-process communication and process control.

The project focuses on important Operating Systems concepts such as **process creation, process states, process identification, inter-process communication (IPC), signal handling, synchronization, and process termination**. It also applies system programming concepts and Linux system calls such as `fork()`, `kill()`, `wait()`, and signal handlers.

The proposed solution consists of a controller process and multiple worker processes. The controller creates and monitors processes and sends predefined signals to perform specific actions. Each worker process registers suitable signal handlers and responds to received signals according to its assigned task. The system can demonstrate operations such as starting, pausing, resuming, communicating with, and safely terminating processes. Process activities and signal events can also be logged for monitoring and analysis.

The expected outcome is a functional Linux-based system that clearly demonstrates how processes communicate and respond to signals. The project provides practical knowledge of Linux process management, signal-based IPC, system calls, and kernel-level process control while helping students understand how multi-process applications can be efficiently managed.

2. Problem Statement

In Linux, multiple processes need to communicate and coordinate efficiently. Managing processes such as **starting, pausing, resuming, and terminating** can be difficult without a simple control mechanism.

This project addresses the problem by using **Linux signals for process communication and control**. It demonstrates how one process can send signals to another and how the receiving process responds using signal handlers.

The project is relevant to **Operating Systems and Systems Programming** as it demonstrates **process management, IPC, synchronization, system calls, and signal handling** in a practical Linux environment.

3. Objectives

1. **Implement process communication** using Linux signals.
2. **Control processes** through start, pause, resume, and termination signals.
3. **Demonstrate signal handling** using Linux system calls.
4. **Understand process management and IPC** concepts in Operating Systems.

4. Proposed Methodology

The project will be designed and implemented in **Linux using C and system calls**. A **controller process** will create and manage multiple worker processes using `fork()`.

Flow:

Create Processes → Register Signal Handlers → Send Signals → Process Responds → Monitor Status → Terminate

- **Process Creation:** `fork()` creates worker processes.
- **Signal Handling:** Signals are registered for different actions.
- **Process Control:** `kill()` sends signals to pause, resume, or terminate processes.
- **Monitoring:** The controller tracks process status and responses.
- **Termination:** `wait()` ensures processes are completed and safely managed.

This methodology demonstrates **Linux process management, IPC, signal handling, and system calls** in a practical manner.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Management	Creates and controls multiple processes.
IPC (Signals)	Enables communication between processes.
<code>fork()</code>	Creates child processes.
<code>Kill()</code>	Sends signals to control processes.
<code>Wait()</code>	Manages and waits for child processes.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520080018	Vennela	Process creation using fork(), worker process implementation and signal handling
2520080009	krishnasree	Controller process, signal-based process control and process monitoring
2520080007	harshitha	Testing, output verification, process termination using wait(), and documentation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Environment for implementing and executing the process communication and control system
C / C++	Programming language for implementing processes, signal handlers, and system calls
fork()	Creates worker/child processes
kill()	Waits for child processes and manages safe termination
wait()	Handles signals and performs corresponding process actions

8. Expected Outcome

The system is expected to successfully create and manage multiple processes using Linux system calls. The controller process will send signals to worker processes to perform actions such as starting, pausing, resuming, communicating, and terminating processes. The system will display process status and signal events, demonstrating effective process communication, signal handling, IPC, process management, and safe process termination in a Linux environment.

9. GitHub Repository

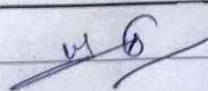
GitHub Repository Link: <https://github.com/krishnasreemokkapati08-max/KLH-AI-DS-2026-10-Linux-Signal-Process-Control>

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signatur

Faculty Name: Raghupathi M

Signature: 

Remarks: _____

Faculty Use

Project Approved: Approved

☒ Yes ☐ No

Date: 9/9/2026

Faculty Signature: [Signature]